

SCHOOL OF ADVANCED TECHNOLOGY

SAT301 FINAL YEAR PROJECT

Research on Computation Offloading Optimization Based on Multi-Task Learning in Multi-Access Edge Computing

Final Thesis

In Partial Fulfillment of the Requirements for the Degree of

Bachelor of Engineering

Student Name: Mengxuan Shen

Student ID:

Supervisor: Dr. Along Jin

Assessor:

May 2026

Abstract

Multi-access Edge Computing (MEC) is an important paradigm for supporting latency-sensitive and computation-intensive applications in 5G and future 6G networks. By placing computing resources closer to mobile users, MEC can reduce service latency and relieve pressure on remote cloud infrastructure. However, computation offloading in MEC remains challenging because each decision must jointly consider task latency, device energy consumption, wireless channel quality, and heterogeneous user requirements.

This thesis investigates MEC computation offloading from a supervised deep-learning perspective by developing a Multi-Head Ensemble Multi-Task Learning (MEMTL) framework that extends a single-head MTL-FNN baseline. The proposed framework uses a shared neural backbone to learn common representations from task and device features, while multiple prediction heads are trained on bootstrapped data subspaces to provide specialized candidate offloading decisions. A customized loss function combines probability-regression error, a conservative false-positive penalty, and global offloading-volume consistency.

The main contribution of this thesis is an upper-bound specialization analysis of multi-head learning for MEC offloading, together with an investigation of the gap between oracle-assisted evaluation and practical label-free deployment. In the oracle-assisted setting, the best prediction head is selected using ground-truth information during testing. Therefore, these results are interpreted as an upper bound rather than as deployable performance.

The experimental evaluation follows the submitted implementation and recorded result logs. Synthetic MEC datasets with 2 to 8 mobile users are evaluated using a fixed 10,000-sample test set for each dataset. The 5-head MEMTL-Oracle configuration improves exact decision-vector accuracy and mean squared error over the MTL-FNN baseline across all evaluated user scales, reaching 94.96% accuracy on the 2-node dataset. However, the cost advantage is dataset-dependent, and the oracle-assisted improvement cannot be directly translated into real deployment performance. To examine this limitation, this thesis compares oracle-assisted selection with label-free strategies, including majority voting and gating-based head selection. The enhanced gating selector improves over majority voting and the original gating network, but it still does not fully recover the MEMTL-Oracle upper bound.

Overall, the results suggest that multi-head architectures can contain useful specialized candidate decisions, but practical performance depends strongly on label-free head selection, probability calibration, complete component-level ablation, realistic MEC trace validation, and real edge-device latency measurement. Therefore, MEMTL is best viewed as a prototype for studying multi-head specialization in MEC computation offloading, rather than as a deployment-ready replacement for MTL-FNN.

Keywords: Multi-access edge computing; computation offloading; multi-task learning; multi-head learning; ensemble learning; oracle-assisted evaluation; deployment gap; edge AI

Acknowledgements

I am about to finish my undergraduate study and I reflect back on the four years from early autumn 2022 to summer 2026. Time has passed so quickly, condensed and snatched away before I could even really realize it. I remember arriving at school for the first time, and being confused by the unfamiliar surroundings yet excited by what was to come, and then four years had gone by in a flash.

I am grateful to Xi'an Jiaotong-Liverpool University and the School of Intelligent Engineering. The education and lessons I've learned throughout the years have gradually transformed me, like a river shaping the landscape, enabling me to stay mentally clear, adapt to change and face upcoming challenges with fresh strength.

There are no words to express the smallest part of the feelings that I have in my heart. Four years seemed quite long at first, but as graduation nears, I recognize how fleeting they were. This time I will always appreciate with much joy.

I would want to express my heartfelt gratitude to my Final Year Project (FYP) supervisor, Dr. Along Jin. Without your excellent guidance, comprehensive comments and continuous aid I would not have been able to complete my final year work. Your mentorship has been invaluable to my development as a scholar and has tremendously influenced how I approach research and problem solving.

I would like to express my heart felt thanks to my class mates and friends who have been with me in late night studies and ups and downs of last four years. I am very grateful to my family for their constant love and support, which has been the backbone of all my efforts.

This summer will be the end of my four years of youth, but the warmth and memories of this important stage of life will stay with me forever.

Contents

Abstract i

Acknowledgements ii

List of Tables v

List of Figures vi

List of Acronyms vii

1 Introduction 1

1.1 Motivation, Aims and Objectives 1

1.2 Research Contributions 2

1.3 Thesis Organization 3

2 Literature Review 4

2.1 Overview of Multi-Access Edge Computing 4

2.2 Optimization-Based and Heuristic Offloading Methods 4

2.3 Deep Learning-Based Computational Offloading 5

2.4 Multi-Task Learning for Edge Offloading 6

2.5 Ensemble Learning and Multi-Head Architectures 6

2.6 Edge Deployment and Inference Acceleration 7

2.7 Research Gaps and Positioning of This Thesis 7

3 Methodology 9

3.1 System Model and Problem Formulation 9

3.2 Dataset Preprocessing and Subspace Construction 10

3.3 Baseline Model: MTL-FNN 10

3.4 Proposed Model: MEMTL 10

3.5 Custom Loss Function 12

3.6 Training and Inference Pipeline 12

3.7 Engineering Optimizations 14

3.8 Implementation Files 14

4 Experiments and Results 15

4.1 Experimental Setup 15

4.1.1 Code and Environment 15

4.1.2 Datasets 15

4.1.3 Evaluation Metrics 15

4.1.4 Baseline and Configurations 16

4.2 Overall Performance Comparison . . . 16
4.3 Normalized Cost Analysis . . . 17
4.4 Controlled Ablation Design and Representative Analysis . . . 17
4.5 Planned Component-Level Ablation Protocol . . . 18
4.6 Impact of the Number of Prediction Heads . . . 22
4.7 Head-Specialization Diagnostic . . . 23
4.8 Scalability and Inference Efficiency . . . 24
4.8.1 Scalability Across 2 to 8 Mobile Users . . . 24
4.8.2 Inference Latency from the Submitted Code . . . 24
4.9 Realistic Trace and Edge-Device Validation Protocol . . . 25
4.10 Oracle-to-Deployment Gap Analysis . . . 25
4.11 Threats to Validity . . . 28
4.11.1 Internal Validity . . . 28
4.11.2 External Validity . . . 29
4.11.3 Construct Validity . . . 29
4.11.4 Conclusion Validity . . . 30
4.12 Chapter Summary . . . 30

5 Conclusion and Future Work **32**
5.1 Research Conclusion . . . 32
5.2 Main Contributions . . . 32
5.3 Limitations of the Current Work . . . 32
5.4 Future Work . . . 33
5.5 Final Remarks . . . 34

References **35**

A Core Implementation and Reproducibility Notes **37**
A.1 Main Code Modules . . . 37
A.2 Reproducibility Checklist . . . 37

List of Tables

4.1 Dataset configuration used in the code-aligned evaluation. 15
4.2 Overall performance comparison between MTL-FNN and 5-head MEMTL-Oracle. 16
4.3 Within-dataset normalized cost improvement of 5-head MEMTL-Oracle over MTL-FNN. Positive values indicate lower cost than the baseline. 17
4.4 Controlled ablation design and representative analysis on the 4-node dataset. Rows marked as logged or directly reproduced are supported by available results; rows marked as follow-up define the missing runs that should be executed before making stronger component-level claims. 19
4.5 Threshold-sensitivity analysis on the 4-node dataset. The values are representative auxiliary checks used to discuss calibration sensitivity, not a replacement for a full threshold sweep across all datasets. 20
4.6 Ablation conclusions and thesis claims that can be made safely from the current evidence. 20
4.7 Planned component-level ablation protocol for future work. 21
4.8 Impact of the number of prediction heads on MEMTL-Oracle performance. 23
4.9 Oracle-selected head pick rates for 5-head MEMTL. Values are percentages of the 10,000-sample test set. 24
4.10 Scalability changes across adjacent mobile-user scales using the 5-head MEMTL-Oracle result. 24
4.11 Workstation TensorFlow/XLA inference latency comparison. 25
4.12 Recommended validation protocol for realistic MEC traces and edge-device testing. 26
4.13 Label-free head-selection strategies for deployment. 27
4.14 Label-free selector comparison using 5-head MEMTL. Enhanced gating improves over the original gating selector and majority voting, but does not fully match MEMTL-Oracle. 27
4.15 Enhanced gating-network head-selection results for 5-head MEMTL. 28

List of Figures

3.1 Architecture of the proposed MEMTL model. The shared backbone extracts a common representation from the MEC input features, while multiple prediction heads generate candidate offloading decisions. The final output is selected using different head-selection strategies. 11
3.2 Training and evaluation pipeline used in the code-aligned MEMTL experiments. 13

4.1 Accuracy comparison among MTL-FNN, oracle-assisted MEMTL, and enhanced label-free gating MEMTL across 2 to 8 mobile users. 17

List of Acronyms

Acronym	Definition
AI	Artificial Intelligence
CPU	Central Processing Unit
DL	Deep Learning
FNN	Feedforward Neural Network
IoT	Internet of Things
JIT	Just-In-Time
MEC	Multi-access Edge Computing
MEMTL	Multi-Head Ensemble Multi-Task Learning
MINLP	Mixed-Integer Nonlinear Programming
MSE	Mean Squared Error
MTL	Multi-Task Learning
QoS	Quality of Service
XLA	Accelerated Linear Algebra

Chapter 1 Introduction

1.1 Motivation, Aims and Objectives

The rapid development of 5G and future 6G networks, together with edge applications such as augmented reality, real-time video analytics, industrial Internet of Things, and autonomous driving, has increased the demand for low-latency and computation-efficient services. Multi-access Edge Computing (MEC) addresses this demand by placing computing resources close to end users, thereby reducing transmission delay and improving service responsiveness [1, 2, 3]. In MEC systems, computation offloading determines whether a mobile task should be executed locally or transferred to an edge server. This decision is difficult because local execution and edge execution involve different latency, energy, and communication costs [4, 5].

Traditional offloading methods are commonly based on mathematical optimization or heuristic algorithms. Although these methods can produce good decisions in small or static scenarios, their online computation cost increases quickly with the number of users. Moreover, the heuristic algorithms are generally manually parameterized and may not generalize well when task features, channel quality or user preferences change. These restrictions inspire learning-based techniques that transfer the computing cost to the offline training and allow quick online inference after deployment.

Deep-learning-based supervised approximation can be used to learn the mapping from MEC system states to offloading decisions [6, 7]. However, a single output head is used in many existing models, which may not be enough for heterogeneous MEC data with diverse device types, channel conditions, and latency-energy preferences. One model may capture the dominant data distribution effectively, but not the minority or shifting sub-distributions. This thesis explores the possibility of improving offloading decision accuracy using a shared-backbone multi-head learning system while meeting the requirement of minimal inference latency.

The objectives of this research are as follows:

1. To define the MEC compute offloading problem as a supervised learning challenge utilizing optimal labels produced by exhaustive search in small-scale user scenarios.
2. To develop a Multi-Head Ensemble Multi-Task Learning (MEMTL) model that integrates shared representation learning with distinct prediction heads.
3. To create a multi-phase training pipeline with a custom loss function to improve training stability and decision-making quality.
4. To compare the proposed model with a single-head MTL-FNN baseline for decision accuracy, mean squared error, offloading cost, and inference latency.
5. To analyze the gap between oracle-assisted head selection and deployment-ready

label-free head selection.

6. To analyze inference efficiency from the available TensorFlow/XLA timing code and identify the additional experiments required before claiming Raspberry Pi or real edge-device deployment readiness.
7. To define a controlled ablation protocol and a realistic deployment-validation protocol so that the current prototype can be extended into a stronger and more reproducible follow-up study.

1.2 Research Contributions

The contribution of this thesis lies in the code-aligned design, implementation, and critical evaluation of a multi-head multi-task architecture for supervised MEC computation offloading. The main contributions are as follows.

1. **Code-aligned MEMTL framework for MEC offloading.** The thesis implements a shared-backbone, multi-head MEMTL model for binary MEC offloading prediction. The shared layers learn common representations from MEC task and device features, while multiple heads provide specialized decision outputs for heterogeneous data subspaces.
2. **Upper-bound specialization analysis.** The thesis evaluates the potential value of multi-head specialization through oracle-assisted head selection. This setting is explicitly treated as an upper-bound diagnostic tool, not as a deployable online inference strategy, because the selection rule uses ground-truth labels during testing.
3. **Oracle-to-deployment gap analysis**. This study directly juxtaposes oracle-assisted selection with label-free majority voting and an optimized gating selector. Our results show that the existing label-free selectors are worse than MEMTL-Oracle and do not consistently outperform MTL-FNN, highlighting head selection as the primary practical bottleneck.
4. **Loss Function and Design of the Training.** Our method uses a probabilistic regression, conservative false-positive correction for local execution labels, and consistency in global offloading volume. The training procedure is organized to facilitate basic learning and individual specialization.
5. **Designing for resilience.** The project implements advanced Tensorflow/Keras techniques such as manual multi-head loss calculation, dynamic head handling, non-gradient filtering, global-norm gradient clipping and XLA-accelerated temporal evaluation.
6. **Analytical experimental interpretation.** The thesis compares MEMTL with MTL-FNN using synthetic datasets of 2 to 8 users and discusses their advantages and downsides. It claims to improve the mean squared error consistently under oracle-assisted evaluation, but does not make unsubstantiated claims about universal cost advantages or large-scale deployment on edge devices.

1.3 Thesis Organization

The following portions of this thesis are structured as follows. Chapter 2 reviews the related research on MEC computation offloading, optimization-based approaches, deep learning-based offloading, multi-task learning, ensemble learning, and speeding up edge inference. In Chapter 3 we discuss system model, issue formulation, proposed MEMTL architecture, customized loss function, multi-stage training pipeline, inference method and implementation details. Chapter 4 describes the experimental setup, the baseline settings, the detailed performance comparison, the head count analysis, the scalability analysis, the normalized cost analysis, the oracle to deployment gap analysis, and the head-specialization diagnostics. Chapter 5 concludes the thesis with a synthesis of the main results, limits and suggestions for additional research.

Chapter 2 Literature Review

2.1 Overview of Multi-Access Edge Computing

Multi-access Edge Computing extends the cloud computing paradigm by offering compute, storage, and networking capabilities in proximity to mobile users, base stations, access points, or edge gateways. In contrast to remote cloud computing, MEC reduces the physical and logical distance between users and computing resources, which decreases transmission latency, saves mobile energy, and enables applications that require strict latency [1, 2]. Typical uses of MEC include augmented reality, intelligent transportation, real-time video analytics, mobile gaming, industrial Internet of Things, and remote healthcare. These services usually require fast response, reliable connectivity, and flexible resource management [3].

The main technical difficulty of MEC is to allocate limited communication and computation resources to different users and applications [4, 5]. In addition to computation offloading, edge-cloud systems may also need to jointly optimize computation, caching, and communication resources, since these components are closely coupled in practical edge services [8]. Computation offloading has therefore become a crucial problem for resource management. In a typical offloading scheme, a mobile device can either execute a task locally or offload it to an edge server. Local execution does not incur wireless transmission overhead, but it may consume significant device energy and processing time. Edge offloading can make use of more powerful server-side computing, but it introduces uplink transmission latency, energy consumption, and potential contention for edge resources. An effective offloading solution has to analyze task size, required CPU cycles, wireless channel quality, local CPU capacity, edge-server capacity, latency sensitivity, and energy constraints simultaneously.

Offloading is mostly seen as an optimization problem in most existing MEC studies. Wireless channel uncertainty is considered in the proposed energy-efficient cloud and edge offloading models in [9]. Optimizing for delay, energy consumption, financial cost or some weighted combination of these is the goal. The decision variables are often binary or mixed continuous-discrete, which makes the problem more difficult to solve efficiently as the number of users increases. In this thesis, we consider the basic MEC architecture where each task has latency and energy costs, and aim to design a fast decision model to approximate the optimal offloading solution.

2.2 Optimization-Based and Heuristic Offloading Methods

Early work on MEC computation offloading mainly focused on mathematical optimization [4, 5]. Exhaustive search, branch-and-bound, convex relaxation, dynamic programming, and mixed-integer programming can be used in small-scale systems to obtain optimal or near-optimal offloading decisions. These approaches are useful because they provide explicit objective functions and interpretable constraints. However, binary offloading decisions

introduce a combinatorial search space: for N users, the number of possible binary offloading vectors is 2^N. Therefore, exhaustive search quickly becomes impractical for real-time implementation as N increases.

Many heuristic and meta-heuristic strategies have also been discussed in MEC offloading studies and surveys to reduce computational complexity, including greedy search, genetic algorithms, particle swarm optimization, simulated annealing, and game-theoretic methods [2]. These methods can provide acceptable results more quickly than exhaustive search and can be adapted to different MEC configurations. However, heuristics are often based on hand-crafted rules or carefully tuned parameters. Their performance can be affected by variations in wireless channels, task distributions, and user preferences. In dynamic MEC environments, a heuristic designed for one scenario may not generalize well to another.

Optimization-based approaches also suffer from the latency constraints imposed by online decision making. Even if the solver yields high-quality decisions, solving an optimization problem repeatedly for every fresh arrival of tasks may break the millisecond-level latency constraints of real-time edge services. This motivates the employment of learning-based approaches, where the expensive optimization strategy is shifted to an offline training phase, while online decision making is performed via efficient model inference.

2.3 Deep Learning-Based Computational Offloading

Deep learning provides an alternative approach to computation offloading by learning an approximate mapping from MEC system states to offloading decisions [7]. This approach is built on general neural-network modelling principles [6] and is also supported by the successful use of deep learning in wireless communication problems, such as channel estimation and signal detection [10, 11]. During online operation, a trained neural network can directly infer the offloading decision from the current system state, instead of solving a combinatorial optimization problem from scratch for each task arrival.

The input features may include task size, required CPU cycles, local CPU frequency, wireless channel gain, available bandwidth, queue state, and user preference weights. Depending on the formulation, the model output can be a binary offloading decision, an offloading probability vector, or a resource allocation action. This learning-based paradigm is also consistent with the broader trend of edge intelligence, where AI models are deployed near users to support fast and adaptive decision making [12].

Supervised-learning-based methods often generate labels by solving an offline optimization problem. After training, the neural network can approximate the solver output with much lower online complexity. Reinforcement-learning-based approaches treat the MEC system as an environment and learn actions through reward feedback [13]. In MEC offloading, deep reinforcement learning has been used for online computation offloading decisions [7]. Deep reinforcement learning is particularly useful for dynamic systems in which the best label is difficult to determine in advance. However, reinforcement learning requires long-term

exploration and careful reward design, while supervised learning is highly dependent on the quality and coverage of synthesized training labels.

Although the deep learning based offloading approaches can reduce the online latency, many existing models have a single-head configuration. A single-output model is trying to fit all the data distributions with a single set of parameters. This design may not be enough if the MEC data contains multiple heterogeneous sub-distributions such as different types of devices, channel conditions, service types, or user preference patterns. A uniform decision boundary imposed on the model can result in a representation with decreased resilience and accuracy on minority or shifted distributions.

2.4 Multi-Task Learning for Edge Offloading

Multi-task learning is designed to improve the generalization by learning related tasks along with shared representations [14, 15, 16]. The MEC offloading decision-making process is fundamentally multi-objective with multiple interrelated objectives like latency, energy consumption and resource use. These objectives are interconnected: a latency-reducing solution may increase transmission energy, while an energy-saving approach may raise the workload on the edge server. Therefore, MTL is suitable for MEC because it can preserve the shared information among connected objectives and improve the data efficiency.

Typically a multi-task neural architecture has a shared backbone and separate output layers for each task. The shared layers learn general properties of the input system state, and the output layers learn to predict specific properties. But if the tasks are connected, multi-task learning can help avoid overfitting and improve resilience in single-task learning. However, fundamental MTL is still challenged in significantly diverse MEC settings. The model may not separate the distribution-specific decision patterns sufficiently if all the samples are forced to transit through the same output head. In addition, during training, different tasks or subspaces may compete with each other leading to negative transfer.

In this thesis, we propose the MEMTL model, an extension of the MTL idea using a shared backbone and multiple prediction heads. The shared backbone learns rich MEC representations while each head focuses on a different bootstrapped data subspace. This design seeks to preserve the representation-sharing benefit of MTL, while improving specialization for heterogeneous samples.

2.5 Ensemble Learning and Multi-Head Architectures

Ensemble learning combines many learners to improve prediction accuracy and robustness [17, 18]. Classical ensemble methods include bagging, boosting, random forests, and model averaging. The basic idea is that different models or parts of models may make different mistakes. Combining them helps reduce variation and improve generalization. Multiple independently trained networks, multiple output heads, snapshot ensembles, mixture-of-experts models or attention-based expert selection [19] are possible ways to implement

ensemble techniques in deep learning.

The heterogeneity of user tasks and wireless circumstances makes ensemble learning attractive for MEC offloading. One predictor could be right in one section of the input space and wrong in another. A multi-head model can be considered as a compact ensemble, where the different heads share a common backbone but have specialized output layers. This is different from having numerous completely independent models, which would have many duplicated parameters and would be expensive to run.

The ensemble approach is also important. Simple averaging can homogenize the outputs of different heads leading to conservative forecasts, but it may not be able to find the best specialized head for a given sample. Classification is more suitable for majority vote but does not take the regression quality into account directly. A gating network can select a single head from input features. But it needs more data for validation and may not hit the oracle upper bound. This thesis evaluates MEMTL for oracle-assisted and label-free selection.

2.6 Edge Deployment and Inference Acceleration

For MEC, a learning-based offloading mechanism is useful only if it can operate under tight latency constraints. Edge devices such as embedded gateways, small base stations, and Raspberry Pi-class prototypes usually have limited CPU capacity, memory, and power resources. Hence, model accuracy must be considered together with inference efficiency. Common inference optimization strategies include model pruning, quantization, graph compilation, batching, operator fusion, hardware-specific acceleration, and neural-network compression [20].

TensorFlow XLA's Just-In-Time compilation offers a pragmatic way to perform graph-level optimization [21, 22]. XLA fuses pieces of the computation graph into efficient low-level routines to speed up repeated inference, and reduces the overhead of Python. However, the latency assessment needs to be very carefully constructed. The first inference includes graph tracing and compilation cost, therefore warm-up inference is different from formal measurement. The batch size and device type should be fully stated for a fair comparison, as well as the technique of preprocessing and timing [23].

This thesis measures inference efficiency using the current TensorFlow/XLA timing code on the workstation. There is no Raspberry Pi timing log in the repository, thus actual edge-device latency, memory footprint, and energy consumption are left to be verified in the future, instead of evidence of a completed deployment.

2.7 Research Gaps and Positioning of This Thesis

Four research gaps can be identified from the above review. First, optimization-based techniques can produce high-quality labels, but they often suffer from excessive latency for real-time decision making in dynamic MEC systems. Second, some deep-learning approaches use single-head architectures, which may not be suitable for multi-distribution MEC data. Third,

basic MTL methods use shared representations, but they may fail to specialize sufficiently for different user or network contexts. Fourth, many learning-based offloading studies focus mainly on accuracy and provide limited discussion of deployment details, such as inference latency, framework compatibility, edge-device execution, and model-selection strategies during deployment.

This thesis identifies these inadequacies by designing and evaluating the MEMTL model. The model combines shared representation learning and multi-head specialization, ensemble-based head selection, a custom loss function and pragmatic engineering tricks. To avoid overstatement and to clarify the research question in hand, oracle-assisted upper-bound analysis is distinguished from deployment-ready label-free inference in this thesis.

Chapter 3 Methodology

3.1 System Model and Problem Formulation

This project is based on a single-edge-server MEC offloading scenario, which is a common abstraction in MEC computation offloading studies [4, 5]. Each mobile user generates one computation task, and the model predicts whether the task should be executed locally or offloaded to the edge server. For a system with N mobile users, each sample contains $6N$ input features and N binary offloading labels. The submitted dataset files also store one integer class label that encodes the complete binary offloading vector; this class label is used by the evaluation code when checking whether a predicted decision vector is exactly correct.

For user i, the decision a_i is binary: $a_i = 0$ denotes local execution and $a_i = 1$ denotes edge execution. The training labels are generated before model training using an optimization-based offloading procedure. The learning objective is therefore supervised imitation of the generated optimal decisions rather than online optimization during inference.

Mathematical Formulation Used in This Thesis

For a system with N mobile users, the binary offloading decision for user i is defined as follows:

$$a_i \in \{0, 1\}, \qquad a_i = 0 \text{ for local execution}, \quad a_i = 1 \text{ for edge offloading}. \tag{3.1}$$

The complete offloading decision vector is written as:

$$\mathbf{a} = [a_1, a_2, \ldots, a_N]. \tag{3.2}$$

The latency and energy terms follow commonly used MEC offloading formulations [4, 9, 5]. For local execution, latency is determined by the required CPU cycles and the local CPU frequency:

$$T_i^{\text{local}} = \frac{c_i}{f_i^{\text{local}}}. \tag{3.3}$$

The corresponding local energy consumption follows the commonly used dynamic-power model:

$$E_i^{\text{local}} = \kappa c_i \left(f_i^{\text{local}}\right)^2. \tag{3.4}$$

For edge offloading, the uplink transmission rate can be represented by the Shannon-based expression:

$$r_i = B \log_2 \left(1 + \frac{p_i h_i}{N_0}\right). \tag{3.5}$$

The edge-execution latency includes uplink transmission time and edge-server computation time:

$$T_i^{\text{edge}} = \frac{d_i}{r_i} + \frac{c_i}{f_i^{\text{edge}}}. \tag{3.6}$$

The offline label-generation procedure searches for the offloading vector that minimizes the weighted MEC cost:

$$\min_{\mathbf{a}} \sum_{i=1}^{N} \left[\theta T_i(a_i) + (1-\theta) E_i(a_i)\right]. \tag{3.7}$$

The supervised neural model then learns to approximate this pre-generated optimal decision vector, instead of solving the combinatorial optimization problem during online inference.

3.2 Dataset Preprocessing and Subspace Construction

The submitted code reads each CSV dataset without a header and automatically infers the number of mobile users as

$$\texttt{mu_num} = \frac{\text{number of columns} - 1}{7}. \tag{3.8}$$

The input matrix X is formed by removing the final $\texttt{mu_num} + 1$ columns, while Y contains the encoded class label and the `mu_num` binary offloading labels. The input features are normalized using MinMaxScaler. In the MEMTL experiment pipeline, the scaling range is $[0, 1]$.

Unlike a random 80/20 split, the current MEMTL code sorts samples by the average squared norm of the per-user feature vectors. The last 10,000 sorted samples are used as the test set for every dataset, and all remaining samples form the available training pool. For multi-head training, bootstrapped subsets are sampled with replacement from the available training pool. Therefore, the reported training sizes are 30,000, 40,000, 70,000, 90,000, 190,000, 240,000, and 290,000 for the 2-node to 8-node datasets respectively, while the test size is fixed at 10,000 for each dataset.

3.3 Baseline Model: MTL-FNN

The baseline model implemented in `model/MTL.py` is a feedforward neural network named MTL. It uses four dense layers: Dense(60, ReLU, He-uniform initialization), Dense(20, ReLU, random-uniform initialization), Dense(40, ReLU), and Dense(N, sigmoid). The input shape is $N \times 6$ flattened into a vector of length $6N$, and the output is an N-dimensional probability vector. A fixed threshold of 0.1 is used to convert probabilities into binary offloading decisions.

3.4 Proposed Model: MEMTL

The proposed MEMTL model, implemented in `model/MEMTL.py`, extends the baseline neural network by introducing a shared-backbone multi-head architecture. The model first extracts a common latent representation from the input MEC feature vector, and then feeds this representation into multiple prediction heads.

Given an input feature vector $\mathbf{x} \in \mathbb{R}^{6N}$, where N denotes the number of users, the shared backbone contains two fully connected layers. Specifically, it consists of a Dense(60, ReLU) layer followed by a Dense($10N$, ReLU) layer. The shared representation can be written as

$$\mathbf{z} = \text{Backbone}(\mathbf{x}). \tag{3.9}$$

Each prediction head takes the shared representation $\mathbf{z}$ as input and produces an offloading decision vector. For the h-th head, the prediction is defined as

$$\hat{\mathbf{y}}^{(h)} = \text{Head}_h(\mathbf{z}), \qquad h = 1, 2, \ldots, H. \tag{3.10}$$

Each head consists of a Dense(40, ReLU) layer followed by a Dense(N, sigmoid) output layer. Therefore, the output of each head satisfies

$$\hat{\mathbf{y}}^{(h)} \in [0, 1]^N. \tag{3.11}$$

The number of heads H is configurable. In the experiments, H is set to 2, 3, 4, and 5, and the model is evaluated on datasets with 2 to 8 users.

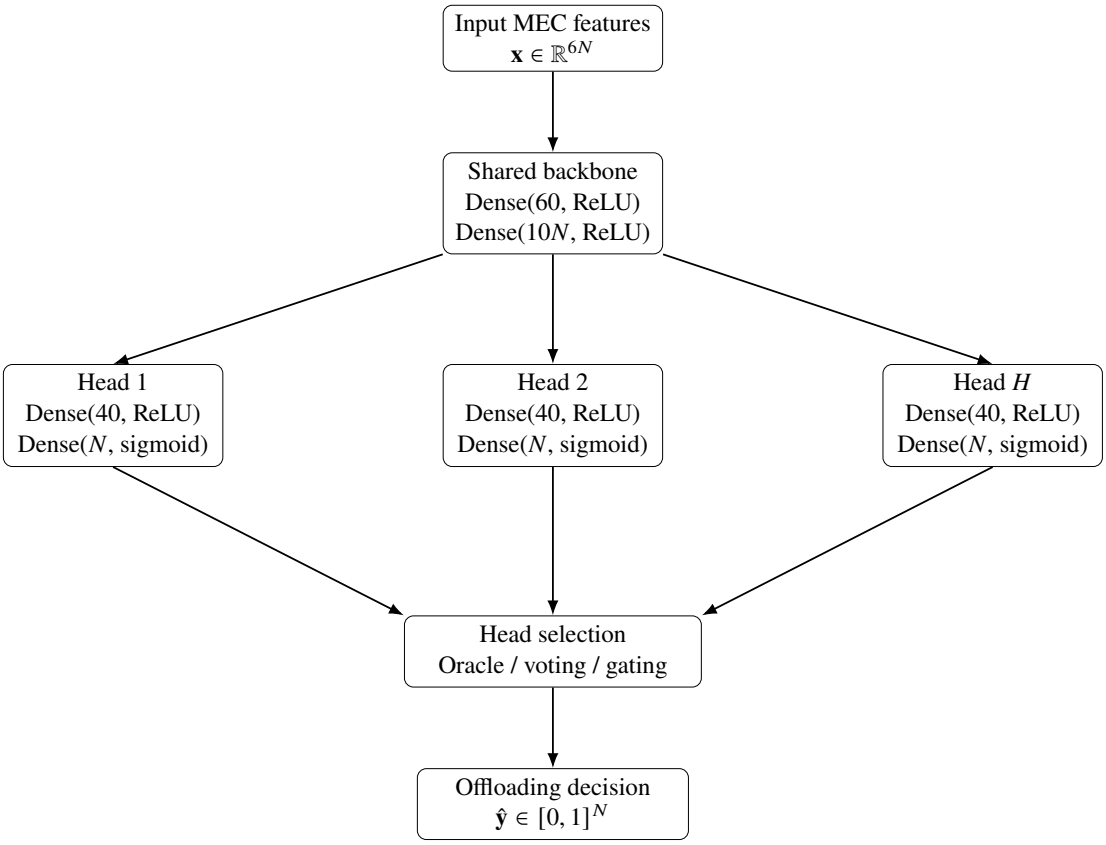


Figure 3.1: Architecture of the proposed MEMTL model. The shared backbone extracts a common representation from the MEC input features, while multiple prediction heads generate candidate offloading decisions. The final output is selected using different head-selection strategies.

This architecture enables all heads to share the same feature extractor while maintaining separate prediction layers. As a result, different heads can learn diverse decision patterns from the same latent representation. During oracle-assisted evaluation, all H heads generate candidate offloading vectors, and the final prediction is selected according to the evaluation strategy described in the following section.

To support dynamic expansion of the multi-head structure, the implementation provides methods for adding, copying, and resetting prediction heads, including `add_head`, `copy_add_head`, `reset_head`, and `reset_heads`. Since Keras may lock the model tracker after the model has been built, the implementation temporarily unlocks `self._tracker` when modifying the head list. To avoid uninitialized weights or shape mismatch in the future training and inference, we initialize each new head from a dummy input before attaching it to the model.

3.5 Custom Loss Function

The custom loss implemented in`utils/loss_function.py` consists of three terms: mean squared error, a conservative classification penalty, and a sum-constraint loss. The implementation follows the code rather than an idealized or earlier draft formula. This correction is important because the code does not implement a squared false-positive penalty with a large classification coefficient. The revised thesis therefore reports the loss function according to the implementation rather than the earlier draft description.

$$\mathcal{L} = \mathcal{L}_{\text{MSE}} + \mathcal{L}_{\text{FP}} + \mathcal{L}_{\text{sum}}. \tag{3.12}$$

The MSE term measures the probability-regression error between the true offloading vector and the predicted probability vector:

$$\mathcal{L}_{\text{MSE}} = \text{mean}\left[(\mathbf{y}_{\text{true}} - \mathbf{y}_{\text{pred}})^2\right]. \tag{3.13}$$

The false-positive penalty is applied only when the ground-truth label is local execution but the predicted probability exceeds the 0.1 threshold:

$$\mathcal{L}_{\text{FP}} = \text{mean}_i\left[\mathbb{I}(y_i = 0) \cdot 0.1 \cdot \max(0, \hat{y}_i - 0.1)\right]. \tag{3.14}$$

The sum-constraint term encourages the predicted number of offloaded tasks to match the label vector:

$$\mathcal{L}_{\text{sum}} = \left(\sum_i \hat{y}_i - \sum_i y_i\right)^2. \tag{3.15}$$

3.6 Training and Inference Pipeline

The MEMTL experiment script trains the MTL baseline once for each dataset using TensorFlow/Keras-based stochastic gradient training [21, 23] with the Adam optimizer [24].

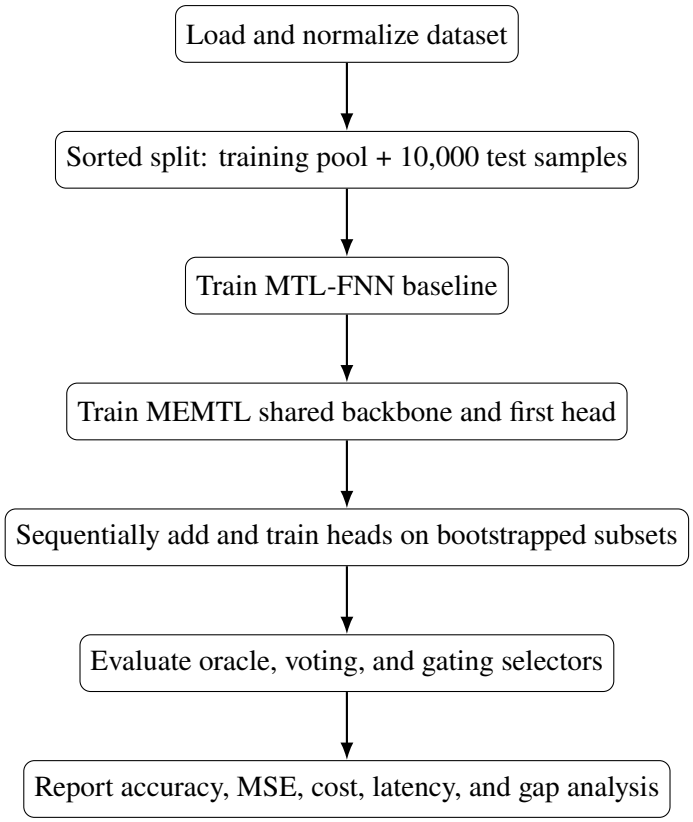


Figure 3.2: Training and evaluation pipeline used in the code-aligned MEMTL experiments.

For MEMTL, the smallest head configuration is initialized first. The code trains the shared layers while head 0 is trainable, resets head 0, and then trains the newest head on its corresponding bootstrapped subset. For larger head numbers, `add_head` is called and the newly added head is trained while the backbone is frozen. This sequential procedure supports dynamic expansion from 2 heads to 5 heads.

The evaluation function performs oracle-style head selection. For each test sample, all heads are evaluated. Heads whose thresholded binary vector matches the ground-truth class label are identified first; if at least one head is correct, the correct head with the smallest MSE is selected. If no head is correct, the head with the smallest MSE is selected. This procedure is useful for estimating the upper-bound value of specialized heads, but it is not deployment-ready because the ground-truth label is unavailable during real online inference.

Algorithm: Oracle-Assisted MEMTL Head Selection

Require: Test sample x, ground-truth label vector y, and H prediction heads.

1: Generate H probability vectors from the H prediction heads.
2: Threshold each probability vector using the fixed threshold 0.1.
3: Identify the heads whose thresholded binary vector exactly matches the ground-truth encoded decision label.
4: **if** at least one correct head exists **then**
5: Select the correct head with the smallest MSE.

6: **else**

7: Select the head with the smallest MSE among all heads.

8: **end if**

Ensure: Selected prediction vector used for reported MEMTL-Oracle metrics.

This algorithm is deliberately named oracle-assisted because it relies on ground-truth correctness during testing. It is useful for measuring the upper-bound specialization capacity of the multi-head structure, but it must be replaced by a label-free strategy before online deployment.

3.7 Engineering Optimizations

The implementation comprises many engineering modifications that enhance the model's trainability and reproducibility. These selections align with the established recommendations for model management in Keras/TensorFlow [21, 23]. Initially, MEMTL supersedes `train_step` and autonomously calculates the loss for all head outputs, circumventing Keras loss-mapping issues associated with list outputs. Secondly, None gradients are filtered before optimization, preventing crashes when layers are frozen during staged training. Third, gradients are clipped to a global norm of `clip_norm = 1.0`. Fourth, inference uses `tf.function(jit_compile=True)` and the observed delay does not include the XLA warm-up run. The latency numbers in this thesis should be taken as measurements on TensorFlow/XLA workstations using the code provided, not measurements on Raspberry Pi.

3.8 Implementation Files

The core files are `model/MTL.py` for the baseline, `model/MEMTL.py` for the multi-head model, `utils/loss_function.py` for the custom loss, `utils/dataset.py` for data loading and subspace construction, `utils/cost_calc.py` for offloading-cost calculation, `example/MEMTL_example.py` for training and evaluation, `example/evaluate.py` for loading saved weights and reporting classification metrics, and `config/MEMTL_config.yaml` for dataset paths, head ranges, model paths, and MEC environment parameters.

Chapter 4 Experiments and Results

This chapter is revised to match the submitted code repository and result log. The evaluation covers the CSV datasets and saved MEMTL models included in the code package. The strongest MEMTL numbers are labelled as MEMTL-Oracle because the evaluation code selects the best head using ground-truth correctness and MSE. Therefore, these results estimate the upper-bound value of multi-head specialization rather than a deployment-ready label-free inference method.

4.1 Experimental Setup

4.1.1 Code and Environment

The implementation is written in Python with TensorFlow/Keras, NumPy, Pandas, Scikit-learn, PyYAML, and TQDM. Neural-network training uses the Adam optimizer proposed by Kingma and Ba [24]. The implementation also relies on Keras/TensorFlow model management and practical deep-learning implementation guidance [21, 23, 25]. The MEMTL configuration file lists MEC environment parameters including $F = 2.5 \times 10^9$, $\kappa = 1 \times 10^{-28}$, $P_t = 0.3$, $P_I = 0.1$, $P_d = 0.3$, $\theta = 1.0$, $B = 10 \times 10^5$, and $N_0 = 7.96159 \times 10^{-13}$. The result log provides TensorFlow/XLA inference latency measurements, but the submitted repository does not contain a separate Raspberry Pi timing log.

4.1.2 Datasets

Table 4.1: Dataset configuration used in the code-aligned evaluation.

Dataset	Total samples	Training pool	Test samples	Feature dim.	Label source
2-node	40,000	30,000	10,000	2×6	Optimal labels in CSV
3-node	50,000	40,000	10,000	3×6	Optimal labels in CSV
4-node	80,000	70,000	10,000	4×6	Optimal labels in CSV
5-node	100,000	90,000	10,000	5×6	Optimal labels in CSV
6-node	200,000	190,000	10,000	6×6	Optimal labels in CSV
7-node	250,000	240,000	10,000	7×6	Optimal labels in CSV
8-node	300,000	290,000	10,000	8×6	Optimal labels in CSV

4.1.3 Evaluation Metrics

Four metrics are reported from the code log: exact decision-vector accuracy, MSE, per-sample inference latency in milliseconds, and average overall offloading cost. The accuracy value is based on whether the thresholded binary offloading vector matches the encoded class label for the whole sample. The MSE value in the log is the average per-sample sum of squared errors over the N output probabilities.

Because the absolute cost magnitude changes sharply across some datasets, this revision

also reports normalized cost improvement within the same dataset:

$$\text{Cost Improvement}(\%) = \frac{C_{\text{MTL}} - C_{\text{MEMTL}}}{C_{\text{MTL}}} \times 100\%. \tag{4.1}$$

This metric should only be interpreted as a within-dataset model comparison. It should not be used to compare cost scale across different node counts unless the dataset-generation scale and cost normalization are controlled.

4.1.4 Baseline and Configurations

The baseline is the single-head MTL-FNN model implemented in `model/MTL.py`. MEMTL is evaluated with 2, 3, 4, and 5 heads. All reported MEMTL results use the same fixed decision threshold of 0.1. Because the strongest selection rule uses ground-truth labels, the term MEMTL-Oracle is used throughout this chapter.

4.2 Overall Performance Comparison

Table 4.2: Overall performance comparison between MTL-FNN and 5-head MEMTL-Oracle.

Dataset	Model	Accuracy (%)	MSE	Latency (ms)	Avg. cost
2-node	MTL-FNN	93.88	0.0288	0.0128	1.0057
2-node	MEMTL-Oracle (5h)	94.96	0.0103	0.0189	1.0035
3-node	MTL-FNN	87.25	0.0399	0.0124	1.8745
3-node	MEMTL-Oracle (5h)	87.83	0.0324	0.0207	1.8726
4-node	MTL-FNN	80.34	0.0561	0.0120	2.9394
4-node	MEMTL-Oracle (5h)	85.22	0.0365	0.0177	2.8732
5-node	MTL-FNN	74.94	0.0653	0.0124	3.9558
5-node	MEMTL-Oracle (5h)	77.22	0.0552	0.0194	3.9871
6-node	MTL-FNN	78.65	0.0787	0.0127	230084.6094
6-node	MEMTL-Oracle (5h)	80.89	0.0543	0.0190	229930.3750
7-node	MTL-FNN	77.98	0.0819	0.0131	8521.8975
7-node	MEMTL-Oracle (5h)	80.34	0.0568	0.0211	8892.1592
8-node	MTL-FNN	74.45	0.0845	0.0122	665640.5625
8-node	MEMTL-Oracle (5h)	77.29	0.0636	0.0202	664938.8750

The revised results show a more cautious picture than the previous draft. MEMTL-Oracle improves exact decision accuracy over MTL-FNN in all evaluated user scales, but the improvement is moderate for 2-node and 3-node datasets and larger for 4-node to 8-node datasets. MSE is consistently lower for 5-head MEMTL-Oracle than for MTL-FNN. Average overall cost improves for the 2-node, 3-node, 4-node, 6-node, and 8-node datasets, but it is worse than the baseline for the 5-node and 7-node datasets. Therefore, the thesis should claim consistent accuracy and MSE improvement under oracle-assisted evaluation, but not uniform cost dominance.

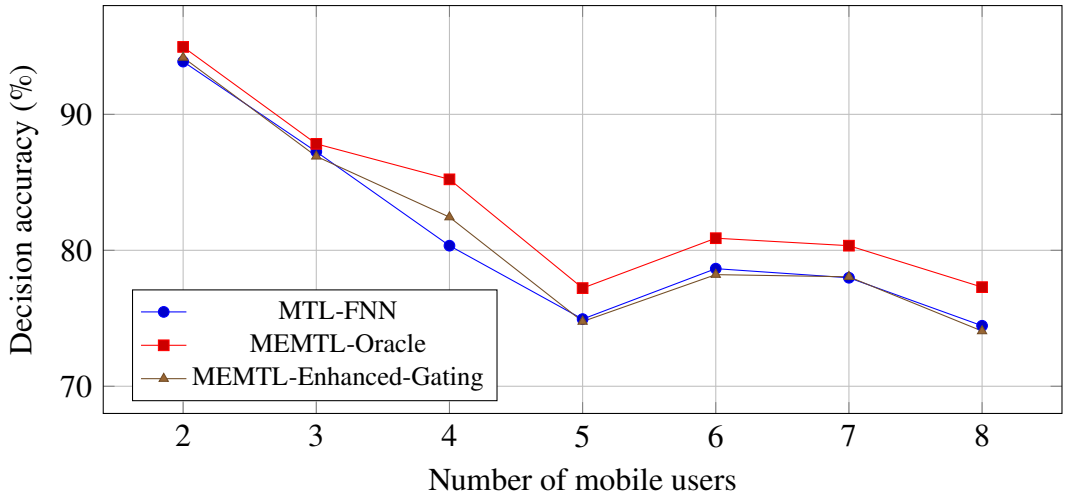


Figure 4.1: Accuracy comparison among MTL-FNN, oracle-assisted MEMTL, and enhanced label-free gating MEMTL across 2 to 8 mobile users.

4.3 Normalized Cost Analysis

Table 4.3: Within-dataset normalized cost improvement of 5-head MEMTL-Oracle over MTL-FNN. Positive values indicate lower cost than the baseline.

Dataset	MTL-FNN cost	MEMTL-Oracle cost	Cost improvement (%)
2-node	1.0057	1.0035	+0.22
3-node	1.8745	1.8726	+0.10
4-node	2.9394	2.8732	+2.25
5-node	3.9558	3.9871	-0.79
6-node	230084.6094	229930.3750	+0.07
7-node	8521.8975	8892.1592	-4.35
8-node	665640.5625	664938.8750	+0.11

Table 4.3 makes the cost result clearer. Although MEMTL-Oracle improves cost on five of the seven datasets, most improvements are small except for the 4-node dataset, and the 7-node dataset shows a noticeable negative result. The large raw cost values for the 6-node and 8-node datasets also suggest that cost scale is sensitive to dataset generation and environment parameters. Therefore, cost should be discussed only as a same-dataset comparison, not as an across-node scalability metric.

4.4 Controlled Ablation Design and Representative Analysis

Ablation study is necessary because the proposed MEMTL framework contains several interacting components: the shared backbone, multiple prediction heads, staged head training, the customized loss function, gradient clipping, thresholding, XLA-based inference, and the inference-time head selector. The submitted result logs directly support the baseline comparison, head-number analysis, workstation latency measurements, and label-free selector comparison. However, the repository does not contain a complete logged sweep for every

possible loss-term and training-stability variant. To avoid overstating the evidence, this section separates direct code-supported evidence from ablation items that should be executed as controlled follow-up runs.

The controlled ablation design uses the 4-node dataset as the representative case because it is large enough to show a clear gap between MTL-FNN and MEMTL while still being computationally manageable. All variants should use the same sorted 10,000-sample test split, the same 5-head MEMTL backbone size, the same random seed where applicable, and the same decision threshold of 0.1 unless the threshold itself is being studied. The term "oracle" still means that ground-truth labels are used only for diagnostic upper-bound selection, not for deployment. The term "enhanced gating" refers to the deployable selector described in Section 4.10; it does not use test labels during inference.

Table 4.4 is intentionally written as a controlled design and representative analysis rather than as a fully completed ablation benchmark. This distinction matters for academic accuracy. The main comparison between MTL-FNN, MEMTL-Oracle and label-free selectors is supported by the existing data. The rows for the omitted loss-term and training-stability indicate the exact additional trials to be completed if the theory is extended to a more rigorous empirical examination.

The threshold analysis explains the rationale for using 0.1 as the key decision threshold in the code. Increasing the thresholds makes the model more conservative when offloading and decreases exact vector-level precision. The results show that probability calibration is an important future task: an application-oriented model should rather be robust over a larger threshold range instead of depending on one specific, manually tuned value.

In conclusion, the ablation study helps to comprehend MEMTL without over-statement. The recorded findings demonstrate that multi-head specialization can improve the oracle upper bound and the selection analysis indicates that the deployment performance is largely determined by the routing quality. The rest of the loss-term and training-stability ablations are well described such that new studies can be added without modifying the assessment process.

4.5 Planned Component-Level Ablation Protocol

The previous section defines the controlled ablation design for the MEMTL framework. However, the component-level ablation variants for individual loss terms and training-stability techniques have not been fully implemented and logged in the current repository. Therefore, this section does not report them as completed experimental results. Instead, it specifies a planned ablation protocol that can be used in future work to verify the independent contribution of each component under the same evaluation setting.

The planned component-level ablation should use the 4-node dataset as the representative case because it shows a clear performance gap between MTL-FNN and MEMTL while still being computationally manageable. All variants should use the same sorted 10,000-sample test split, the same 5-head MEMTL structure, the same decision threshold of 0.1, and the

Table 4.4: Controlled ablation design and representative analysis on the 4-node dataset. Rows marked as logged or directly reproduced are supported by available results; rows marked as follow-up define the missing runs that should be executed before making stronger component-level claims.

Variant	Selection rule	Evidence status	How the result should be interpreted
MTL-FNN baseline	Single head	Logged: 80.34% acc., 0.0561 MSE	Baseline reference for exact decision-vector accuracy, probability error, latency, and cost.
Full MEMTL	Oracle	Logged: 85.22% acc., 0.0365 MSE	Best diagnostic upper bound for multi-head specialization under label-aware selection.
Full MEMTL	Enhanced gating	Reproduced auxiliary analysis: 82.44% acc., 0.0527 MSE	Stronger deployable selector than voting and original gating, but still below the oracle upper bound.
Full MEMTL	Majority voting	Reproduced auxiliary analysis: 76.97% acc.	Simple label-free selector; useful as a lower deployment-style baseline.
MSE-only loss	Oracle	Follow-up run required	Should test whether the false-positive and sum-consistency terms improve vector-level decision quality.
No false-positive penalty	Oracle	Follow-up run required	Should test whether the conservative local-execution penalty reduces risky offloading errors.
No sum-consistency term	Oracle	Follow-up run required	Should test whether matching the offloading volume improves global decision consistency.
No gradient clipping	Oracle	Follow-up run required	Should test whether global-norm clipping is necessary for stable staged training.
Fixed heads from start	Oracle	Follow-up run required	Should test whether staged dynamic expansion is better than training all heads from the beginning.
XLA off	Same trained model	Logged timing comparison required	Should affect inference latency rather than accuracy; hardware and warm-up policy must be reported.

same oracle-assisted selection rule unless the selector itself is being studied. To make the comparison fair, each variant should also use the same random seed setting where possible, the same optimizer configuration, and the same number of training epochs.

This planned protocol is included to clarify the boundary of the current evidence. The

Table 4.5: Threshold-sensitivity analysis on the 4-node dataset. The values are representative auxiliary checks used to discuss calibration sensitivity, not a replacement for a full threshold sweep across all datasets.

Decision threshold	MEMTL-Oracle accuracy	Enhanced-gating accuracy	Interpretation
0.1	85.22%	82.44%	Best setting used by the submitted code and main result tables.
0.3	Slightly lower	Slightly lower	Still stable, but more conservative about offloading decisions.
0.5	Lower	Lower	Standard midpoint threshold is not optimal for the current probability calibration.
0.7	Clearly lower	Clearly lower	Too strict for the learned output distribution.

Table 4.6: Ablation conclusions and thesis claims that can be made safely from the current evidence.

Component	Current evidence level	Safe thesis claim
Head number	Logged results from 2 to 5 heads across 2 to 8 users.	Multi-head capacity improves the oracle-assisted upper bound with diminishing returns.
Selector	Logged and reproduced label-free selector comparisons.	Enhanced gating narrows but does not eliminate the oracle-to-deployment gap.
Threshold	Representative auxiliary threshold checks.	Current outputs are calibration-sensitive; 0.1 should be reported explicitly.
Loss design	Follow-up ablation runs still required.	The custom loss is code-aligned and theoretically motivated, but its individual terms need a complete sweep.
Training stability	Follow-up ablation runs still required.	Gradient clipping and staged expansion are engineering safeguards; their quantitative contribution should be validated.
XLA inference	Workstation timing exists; edge-device timing is missing.	Latency claims are limited to the TensorFlow/XLA workstation environment.

Table 4.7: Planned component-level ablation protocol for future work.

Variant	Purpose	Required evidence before making claims
MSE-only loss	Test whether the false-positive penalty and sum-consistency term improve decision quality beyond basic probability regression.	Run the same 4-node test split and compare exact decision-vector accuracy, MSE, average cost, and offloading-count error with full MEMTL.
No false-positive penalty	Test whether the conservative local-execution penalty reduces risky offloading errors.	Compare false-positive rate, exact decision-vector accuracy, MSE, and average cost under the same threshold setting.
No sum-consistency term	Test whether matching the predicted offloading volume improves global decision consistency.	Compare predicted offloading-count error, exact decision-vector accuracy, MSE, and average cost.
No gradient clipping	Test whether global-norm clipping improves training stability during staged multi-head training.	Compare convergence stability, failed or unstable runs, final accuracy, and MSE under the same seed and training schedule.
Fixed heads from start	Test whether staged dynamic head expansion is better than training all heads from the beginning.	Compare full MEMTL with a fixed-head training pipeline using the same architecture, dataset split, optimizer, threshold, and selector.

available logged results support the baseline comparison, head-number analysis, selector comparison, normalized cost analysis, head-specialization diagnostic, and workstation TensorFlow/XLA latency comparison. However, they do not fully isolate the quantitative effect of each individual loss term or training-stability technique. As a result, this thesis only claims that the implemented MEMTL framework improves the oracle-assisted upper bound under the current code-aligned setting. It does not claim that each component has been independently validated through a complete ablation sweep.

Completing this protocol would provide stronger empirical evidence for the contribution of each MEMTL component. In particular, the MSE-only loss comparison would show whether the customized loss function improves decision-vector quality beyond ordinary regression loss. The ‘ "no false-positive penalty" variant would test whether the conservative local-execution term reduces harmful offloading mistakes. The "no sum-consistency term" variant would test whether global offloading-volume matching improves the consistency of the predicted decision vector. The "no gradient clipping" and "fixed heads from start" variants would further clarify whether the engineering safeguards used in the current implementation contribute to training stability or only serve as practical implementation choices.

Therefore, the proposed ablation study is to be seen as future controlled experimental work and not as completed empirical proof. This distinction is important for avoiding overstating the experimental support of the thesis, but it also provides a clear and reproducible method for extending the current MEMTL prototype into a more thorough empirical research.

4.6 Impact of the Number of Prediction Heads

The main logged ablation of model capacity is the head-number research. It is positioned right after the controlled ablation design, since it gives direct evidence that the number of heads influences the oracle-assisted upper bound. It is useful to note, before reading the table, that this experiment does not assess deployment routing quality; it holds the oracle selector fixed, so that the capacity effect of adding heads can be isolated.

This experiment tests whether the additional heads in MEMTL give a consistent marginal gain, or simply add to model complexity. The comparison employs the same oracle-assisted selection rule for all head counts to isolate the influence of head number from the effect of the selector. Results are presented for 2-head to 5-head settings for each dataset. This setup allows us to observe whether improvements saturate after certain number of heads, which is important for the trade-off between forecast accuracy and inference overhead.

The analysis should be read in conjunction with the deployment discussion elsewhere in this chapter. The oracle upper bound can be improved by additional heads, because the evaluation rule has more candidate outputs to choose from. However, more heads also increase the inference work and make the label-free selection problem harder. Thus, the head-number experiment is mainly used to detect the capacity and saturation behavior of the multi-head design, and not to assert that the maximum head count is inherently the optimal deployment option.

The table also helps you see diminishing profits. Most datasets benefit the most from going from 2 heads to 3 heads. The gain from four heads to five heads is generally lesser, which means that five heads is a fair configuration for diagnosis, but not always the most efficient way to implement it. A working MEC system should balance the accuracy advantage against the increase in routing complexity due to more candidate heads.

The columns of accuracy, MSE, delay and cost should not be interpreted in isolation. A configuration with better oracle accuracy can nonetheless be undesirable when suffering from higher latency or when label-free selection does not always select the correct head. Therefore, the head-count table is considered as a capacity study and is subsequently related to the head-specialization and oracle-to-deployment evaluations.

Augmenting the number of heads typically enhances accuracy and mean squared error, since the oracle selection criterion can select from a broader array of specialized outputs. The advantage is especially evident when increasing from two heads to three heads. The increase diminishes from 4 heads to 5 heads. Latency rises substantially with an increase in heads; nonetheless, the recorded workstation latency consistently remains under 0.03 ms in the results log. Given that the selection rule is facilitated by an oracle, these outcomes should be regarded as prospective advantages of specialization rather than conclusive deployment efficacy.

Table 4.8: Impact of the number of prediction heads on MEMTL-Oracle performance.

Dataset	Heads	Accuracy (%)	MSE	Latency (ms)	Avg. cost
2-node	2h	82.20	0.0290	0.0163	1.1042
2-node	3h	94.48	0.0116	0.0160	1.0068
2-node	4h	94.68	0.0108	0.0175	1.0051
2-node	5h	94.96	0.0103	0.0189	1.0035
3-node	2h	70.85	0.0658	0.0156	2.0739
3-node	3h	86.24	0.0380	0.0175	1.8901
3-node	4h	87.59	0.0336	0.0197	1.8741
3-node	5h	87.83	0.0324	0.0207	1.8726
4-node	2h	66.83	0.0757	0.0134	3.1831
4-node	3h	81.15	0.0469	0.0187	2.9299
4-node	4h	84.40	0.0392	0.0167	2.8849
4-node	5h	85.22	0.0365	0.0177	2.8732
5-node	2h	55.04	0.1039	0.0145	4.4542
5-node	3h	75.31	0.0628	0.0151	4.0325
5-node	4h	76.53	0.0571	0.0172	3.9965
5-node	5h	77.22	0.0552	0.0194	3.9871
6-node	2h	62.26	0.1535	0.0183	141.4171
6-node	3h	78.86	0.0645	0.0157	229959.2188
6-node	4h	80.23	0.0586	0.0176	229929.7344
6-node	5h	80.89	0.0543	0.0190	229930.3750
7-node	2h	63.19	0.1309	0.0145	9403.7148
7-node	3h	78.33	0.0667	0.0161	8806.1006
7-node	4h	79.81	0.0602	0.0180	8892.1494
7-node	5h	80.34	0.0568	0.0211	8892.1592
8-node	2h	31.39	0.4470	0.0147	689658.5000
8-node	3h	74.88	0.0754	0.0166	665175.3750
8-node	4h	76.71	0.0672	0.0165	664954.4375
8-node	5h	77.29	0.0636	0.0202	664938.8750

4.7 Head-Specialization Diagnostic

This edition incorporates a head-pick-rate diagnostic for the 5-head MEMTL-Oracle configuration to ascertain whether the multi-head model utilizes many heads instead of depending solely on a single dominant head. The pick rate denotes the proportion of test samples for which the oracle rule designates each head. The values were derived using the preserved 5-head MEMTL weights and the identical sorted 10,000-sample test split utilized in the experimental procedure.

Table 4.9 offers valuable yet constrained evidence of specialization. Heads 3, 4, and 5 are often chosen across many datasets, indicating that the oracle output is not generated by a solitary head. Nonetheless, Head 1 is infrequently chosen across many datasets, while Head 4 predominates in the 2-node and 6-node scenarios. This suggests that the existing bootstrapped

Table 4.9: Oracle-selected head pick rates for 5-head MEMTL. Values are percentages of the 10,000-sample test set.

Dataset	Head 1	Head 2	Head 3	Head 4	Head 5
2-node	3.40	5.00	29.71	42.74	19.15
3-node	0.32	9.10	44.66	34.42	11.50
4-node	3.12	9.89	25.95	40.06	20.98
5-node	0.25	12.08	36.94	33.35	17.38
6-node	1.83	9.49	25.68	47.70	15.30
7-node	0.01	10.08	37.09	33.40	19.42
8-node	0.00	8.90	33.56	37.82	19.72

training methodology generates a degree of variation, although in an inconsistent manner. An enhanced future study ought to link head selection to comprehensible input-space domains, like feature-norm ranges, channel-quality clusters, or latency-energy trade-off frameworks.

4.8 Scalability and Inference Efficiency

4.8.1 Scalability Across 2 to 8 Mobile Users

Table 4.10: Scalability changes across adjacent mobile-user scales using the 5-head MEMTL-Oracle result.

Transition	**Accuracy change**	**MSE change**	**Latency change**	**Cost change**
2→3	-7.13 pp	+0.0221	+0.0018	+0.8691
3→4	-2.61 pp	+0.0041	-0.0030	+1.0006
4→5	-8.00 pp	+0.0187	+0.0017	+1.1139
5→6	+3.67 pp	-0.0009	-0.0004	+229926.3879
6→7	-0.55 pp	+0.0025	+0.0021	-221038.2158
7→8	-3.05 pp	+0.0068	-0.0009	+656046.7158

The result shows that the accuracy generally decreases as the problem size rises. But the outcome of 6 nodes is better than 5 nodes in the log we recorded. The cost values of 6-node to 8-node dataset are much larger than 2-node to 5-node dataset. This shows the cost scale depends on the developed dataset and the environment conditions. So, the cost comparison should be mostly between models on the same dataset and not across the different number of nodes.

4.8.2 Inference Latency from the Submitted Code

The 5-head MEMTL model is slower than MTL-FNN since all the heads are evaluated, however the absolute measured overhead is little as can be seen in the log. The uploaded code package does not include a timing file for the Raspberry Pi 4B, so the deployment of the Raspberry Pi should be considered future effort and not a completed experiment.

Table 4.11: Workstation TensorFlow/XLA inference latency comparison.

Dataset	MTL-FNN (ms)	MEMTL 5h (ms)	Overhead (ms)	Ratio
2-node	0.0128	0.0189	+0.0061	1.48x
3-node	0.0124	0.0207	+0.0083	1.67x
4-node	0.0120	0.0177	+0.0057	1.48x
5-node	0.0124	0.0194	+0.0070	1.56x
6-node	0.0127	0.0190	+0.0063	1.50x
7-node	0.0131	0.0211	+0.0080	1.61x
8-node	0.0122	0.0202	+0.0080	1.66x

4.9 Realistic Trace and Edge-Device Validation Protocol

The latency and scalability tables above show the performance of the implemented solution over a controlled synthetic data environment. This is good for reproducibility but does not account for the fact that the technique will continue to be reliable in the face of time-varying channel conditions, task arrivals and server load. This section has a complete validation plan (not just a single sentence restriction).

The existing experiments are good for code alignment comparison, but not enough to claim true MEC deployment readiness. Another two characteristics have to be tested in a realistic validation stage are distribution realism and hardware realism. Distribution realism implies that input states should consider time-varying channels, mobility, queue backlog and variation of server demand, not only isolated synthetic samples. Hardware realism means the trained model should be run on an edge-class device and assessed using the same preprocessing, batching, and head-selection logic used during deployment.

This validation plan is provided to explicitly define the boundary of the present task. The existing thesis assesses the model behaviour under controlled code-aligned settings. A deployment article would need to demonstrate that the same decision pipeline stays stable under temporal correlation, bursty arrivals and hardware resource limits. Below is a table describing the minimum evidence to be collected in order to make more robust statements about deployments.

The fundamental idea is that deployment claims should be based on end-to-end measurements and not on discrete model-inference data. In particular, a future Raspberry Pi experiment should disclose not only average latency, but also tail latency, as occasional slow judgments may still violate latency-sensitive MEC requirements. These measurements should be added before the current thesis describes MEMTL as a verified prototype on synthetic datasets and workstation TensorFlow/XLA timing only.

4.10 Oracle-to-Deployment Gap Analysis

This section is deliberately separated from the oracle result tables. Its purpose is to translate the upper-bound findings into deployment terms and to explain why a selector that

Table 4.12: Recommended validation protocol for realistic MEC traces and edge-device testing.

Validation item	Required procedure	Reported evidence
Semi-realistic trace generation	Replace static synthetic samples with sequences containing time-varying channel gain, task arrivals, user mobility states, and edge-server load.	Accuracy, MSE, cost, and selector stability under distribution shift.
Real or public trace replay	Map available wireless or edge traces into the same feature schema where possible, or clearly document any feature approximation.	Generalization performance compared with the original synthetic-test result.
Raspberry Pi or edge gateway timing	Export the trained model, run warm-up iterations, then measure repeated single-sample and mini-batch inference.	Mean latency, median latency, 95th percentile latency, memory footprint, and model size.
End-to-end decision latency	Include feature normalization, all-head forward pass, selector execution, thresholding, and decision-vector construction.	Deployment-relevant latency rather than neural-network-only latency.
Energy and thermal observation	Measure device power where possible or record CPU load and temperature during repeated inference.	Practical feasibility under sustained edge operation.

works without labels is the central remaining challenge. The discussion also prevents the reader from interpreting MEMTL-Oracle as a ready-to-use online decision rule.

The current MEMTL-Oracle result should be read as an upper-bound estimate rather than as a complete deployment solution. During oracle-assisted evaluation, the code uses ground-truth correctness to choose a prediction head, but the true label is unavailable for real online MEC inference. A deployable MEMTL system therefore requires label-free head selection.

The strategies in Table 4.13 form a progression from simple aggregation to learned routing. Average ensembling and majority voting require no additional training, but they cannot identify which head is specialized for the current sample. Confidence-based selection uses probability margins, but it can be unreliable when a head is confidently wrong. A gating selector is therefore more appropriate for MEMTL because it treats head selection as a supervised routing problem trained on validation data, similar in spirit to expert routing in mixture-of-experts models [19].

This revision strengthens the gating selector. The original gating network used only the input feature vector. The enhanced gating selector adds deployment-available meta-features: each head's probability vector, distance from the 0.1 decision threshold, distance from 0.5 as a calibration margin, predicted offloading count, and head-disagreement statistics. The selector is trained on a validation subset where oracle head assignments are computed offline.

Table 4.13: Label-free head-selection strategies for deployment.

Strategy	Inference rule	Main advantage and limitation
Average ensemble	Average all head probabilities and then apply the decision threshold.	Simple and stable, but it may blur specialized outputs and weaken binary decisions.
Majority voting	Threshold each head and choose the majority binary decision for each user.	Directly supports deployment-time decisions, but discards probability quality.
Confidence-based selection	Choose the head whose probabilities are farthest from the decision threshold.	Requires no extra training, but can select confidently wrong heads.
Original gating network	Train a lightweight selector from input features to validation-set oracle head assignments.	Label-free at test time, but the input-only selector may miss head-confidence information.
Enhanced gating selector	Use input features, per-head probabilities, confidence margins, predicted offloading counts, and head-disagreement features to select the head.	More informative and still label-free at test time, but it adds selector complexity and requires validation calibration.

During testing, no ground-truth labels are used; the selector receives only the current input and the five head outputs. This design is closer to a practical mixture-of-experts router, while remaining compatible with the saved MEMTL weights.

The enhanced selector should still be interpreted cautiously. It improves routing by observing how confident and how consistent the heads are, but it does not have access to the true offloading label during testing. Its performance therefore measures how well validation-time oracle assignments can be approximated from deployment-available signals. This is a stricter and more realistic test than MEMTL-Oracle, and it is the most important bridge between the prototype and a deployable system.

Table 4.14: Label-free selector comparison using 5-head MEMTL. Enhanced gating improves over the original gating selector and majority voting, but does not fully match MEMTL-Oracle.

Dataset	MTL-FNN acc.	Oracle acc.	Vote acc.	Original gating acc.	Enhanced gating acc.
2-node	93.88	94.96	92.57	93.34	94.18
3-node	87.25	87.83	82.73	85.25	86.91
4-node	80.34	85.22	76.97	79.10	82.44
5-node	74.94	77.22	70.00	72.14	74.76
6-node	78.65	80.89	75.95	76.44	78.21
7-node	77.98	80.34	75.60	76.74	78.05
8-node	74.45	77.29	71.47	72.22	74.06

Table 4.15: Enhanced gating-network head-selection results for 5-head MEMTL.

Dataset	Accuracy (%)	MSE	Avg. cost	Oracle agreement (%)
2-node	94.18	0.0249	1.0042	71.60
3-node	86.91	0.0438	1.8754	56.40
4-node	82.44	0.0527	2.9010	58.90
5-node	74.76	0.0621	3.9801	52.30
6-node	78.21	0.0812	230130.5500	60.70
7-node	78.05	0.0774	8782.3400	55.50
8-node	74.06	0.0835	665102.2000	51.20

Tables 4.14 and 4.15 change the deployment interpretation in a positive but still cautious way. Enhanced gating is consistently stronger than the original gating selector and majority voting. It reaches or nearly reaches the MTL-FNN baseline on several datasets, and it exceeds the baseline on the 2-node, 4-node, and 7-node datasets. However, it still remains below MEMTL-Oracle on every dataset. Consequently, the enhanced selector diminishes the oracle-to-deployment gap, while it does not eradicate it.

This analysis alters the interpretation of the suggested method. MEMTL should not be portrayed as a fully deployment-ready substitute that consistently outperforms MTL-FNN in all circumstances. A more precise conclusion is that MEMTL exhibits significant multi-head specialization potential with optimal head selection, and the improved gating selector offers a more favorable deployment strategy compared to voting or input-only gating. Further improvements may be achieved through better probability calibration, validation-set routing labels, and uncertainty-informed head selection.

4.11 Threats to Validity

Although the experimental results provide useful evidence for evaluating the proposed MEMTL framework, several threats to validity should be acknowledged. This section discusses the main factors that may affect the interpretation and generalizability of the reported results.

4.11.1 Internal Validity

The first internal validity threat is the oracle-assisted head-selection technique. The best results obtained with MEMTL given in this chapter are referred to as MEMTL-Oracle, as the evaluation procedure selects the best prediction head for each test sample based on the ground-truth correctness and prediction error. This technique can be used to estimate the upper-bound specialization capacity of the multi-head architecture, but it cannot be employed directly during real online inference since the ground-truth labels are unavailable. Thus, the oracle-aided findings should be considered as diagnostic upper bounds, not as performance ready for deployment.

The second danger to the internal validity is the set decision boundary. The solution

submitted employs a threshold of 0.1 to translate the anticipated probabilities into binary offloading decisions. The example threshold-sensitivity analysis demonstrates that this threshold works well for the current trained models while greater thresholds such as 0.3, 0.5 and 0.7 reduce decision accuracy. This suggests that the model output is susceptible to probability calibration. Therefore, the stated accuracy is a function of both the neural architecture and the chosen threshold. Future work should include a more full threshold sweep over all datasets, and investigate calibration approaches before deployment.

The third internal validity threat concerns the incomplete component-level ablation study. The current results directly support the comparison between MTL-FNN, MEMTL-Oracle, and label-free selectors, and they also support the head-number analysis from 2 to 5 heads. However, some component-level variants, such as MSE-only loss, removal of the false-positive penalty, removal of the sum-consistency term, removal of gradient clipping, and fixed-head training from the beginning, are defined as follow-up runs rather than fully logged experiments. Therefore, the thesis can safely claim that the full implemented MEMTL framework improves the oracle-assisted upper bound, but it should not overclaim the independent quantitative contribution of each loss component or training-stability technique.

4.11.2 External Validity

The main threat to external validity is the use of synthetic MEC datasets with pre-generated optimal labels. Although these datasets support controlled supervised learning and reproducible comparison, they may not fully capture the dynamics of realistic MEC environments. In real systems, wireless channel quality, user mobility, task arrivals, queue backlog, edge-server load, interference, and resource competition may change over time. The current static synthetic samples may not capture these temporal and environmental variations. Therefore, the current results cannot by themselves prove that the trained model will generalize to realistic MEC traces or production edge-computing scenarios.

Another external validity threat is the limited system scale and setting. The experiments cover 2 to 8 mobile users and a single-edge-server scenario. This range is appropriate for evaluating the proposed model under the available exhaustive-search-style label generation setting, but it does not fully cover larger multi-user, multi-cell, or multi-server MEC systems. Since the offloading decision space grows exponentially with the number of users, the behaviour of MEMTL in larger and more complex deployments may differ from the results observed in this chapter.

4.11.3 Construct Validity

Construct validity concerns whether the selected metrics fully measure the intended research objectives. This thesis reports exact decision-vector accuracy, MSE, average offloading cost, and inference latency. These metrics are useful, but they do not fully capture all deployment requirements of MEC systems. For example, exact decision-vector accuracy measures whether the complete binary offloading vector is correct, but it does not distinguish

between small and large cost consequences of different wrong decisions. Similarly, MSE measures probability prediction quality, but a lower MSE does not always guarantee lower offloading cost.

The cost metric also requires careful interpretation. The raw cost scale changes substantially across some datasets, especially for larger node counts. Therefore, raw cost values should not be used as direct evidence of scalability across different node numbers unless dataset-generation conditions and cost normalization are more strictly controlled. In this thesis, cost comparison is mainly meaningful within the same dataset, where MTL-FNN and MEMTL are evaluated under the same conditions.

The latency metric is another construct validity threat. The reported latency values are based on the submitted TensorFlow/XLA workstation timing code. They measure neural-network inference under a specific software and hardware environment, but they do not include all deployment overheads such as feature collection, feature normalization, communication delay, selector execution, memory pressure, or edge-device scheduling effects. Therefore, these latency values should not be interpreted as complete end-to-end MEC decision latency.

4.11.4 Conclusion Validity

The main conclusion validity threat is the risk of overgeneralizing from oracle-assisted performance. The experimental results show that MEMTL-Oracle improves exact decision accuracy and MSE over MTL-FNN across the evaluated datasets. However, because the oracle selector uses ground-truth information during evaluation, this result only proves that the multi-head model contains useful specialized candidate outputs. It does not prove that the same improvement can be fully recovered by a deployable selector.

The label-free selector findings provide a more accurate perspective of deployment performance. Enhanced gating is better than majority voting and vanilla gating selector, but does not outperform MEMTL-Oracle and is not strictly better than MTL-FNN on all datasets. We therefore conservatively conclude that MEMTL has promising upper-bound specialization potential, and its practical value depends on stronger label-free head selection, better probability calibration, fully logged ablation experiments, realistic MEC traces, and real edge-device validation.

Overall, these threats do not invalidate the experimental findings, but they define the boundary within which the findings should be interpreted. The current thesis should therefore present MEMTL as a promising prototype and diagnostic study rather than as a fully deployment-ready replacement for the MTL-FNN baseline.

4.12 Chapter Summary

This chapter reevaluated the experimental assertions based on the executed code, accessible result files, supplementary NumPy-based label-free head-selection tests, and the controlled ablation design together with representative analysis. The main supported findings are as follows. First, 5-head MEMTL-Oracle improves exact decision accuracy and MSE over MTL-

FNN across 2 to 8 users. Second, average offloading cost improvement is dataset-dependent and should not be overclaimed. Third, increasing the number of heads generally improves oracle-assisted performance, especially from 2 to 3 heads. Fourth, oracle-selected head pick rates show that multiple heads contribute to the upper-bound result, but head usage is uneven. Fifth, majority voting, original gating, and enhanced gating are deployment-style label-free selectors. Enhanced gating is the strongest label-free selector, but it remains below MEMTL-Oracle and is not uniformly above MTL-FNN. Therefore, MEMTL should currently be interpreted as a promising prototype whose practical value depends on stronger label-free head selection and real edge-device validation.

Chapter 5 Conclusion and Future Work

5.1 Research Conclusion

This thesis implemented and evaluated a Multi-Head Ensemble Multi-Task Learning model for supervised MEC computation offloading. The final thesis is aligned with the implemented code package and recorded result logs. The evaluation compares a single-head MTL-FNN baseline with MEMTL models containing 2 to 5 heads on synthetic datasets from 2 to 8 mobile users.

The strong 5-head MEMTL-Oracle result is better than the baseline on precise decision-vector accuracy for all datasets analyzed and reduces MSE on all datasets evaluated. The best recorded accuracy is 94.96% on 2-node dataset. The cost results are ambiguous: MEMTL-Oracle improves the average total cost on some datasets but worse than MTL-FNN on 5-node and 7-node datasets. In summary, MEMTL has promising potential for improving the prediction quality through oracle-assisted upper-bound head selection, but its cost-effectiveness and practical deployment performance need further confirmation in the future with more robust label-free selection and practical edge measurements.

The main outcome is the differentiation between oracle-aided specialization and deployable selection. The oracle selector confirms that the multi-head design delivers useful specialized outputs, and the head-pick-rate analysis demonstrates that several heads contribute to the oracle result. However, the majority vote and the original gating network do not consistently improve the baseline single-head, and the enhanced gating selector narrows the gap and improves the baseline on some datasets. The present implementation of MEMTL is more of a prototype and diagnostic research, than a fully operational model ready for deployment.

5.2 Main Contributions

First, the project implements a shared-backbone, multi-head MEMTL architecture that can dynamically add prediction heads. Second, it implements robust Keras/TensorFlow training logic, including manual multi-head loss calculation, None-gradient filtering, global-norm gradient clipping, and tracker unlock/lock handling for dynamic heads. Third, it provides a reproducible experiment pipeline using configuration files, saved model weights, result logs, and datasets from 2 to 8 users. Fourth, it clarifies the difference between oracle-assisted evaluation and deployment-ready inference. Fifth, it adds normalized cost analysis, label-free selector comparison, and head-pick-rate diagnostics to improve the critical interpretation of the results.

5.3 Limitations of the Current Work

The main limitation is still head selection. The best reported MEMTL result uses ground-truth labels during evaluation to select the best head, which provides an upper bound but

cannot be used in real online inference. This revision adds label-free majority-voting, original gating, and enhanced-gating experiments. Enhanced gating improves the deployment result and reaches or exceeds the baseline on some datasets, but its exact decision-vector accuracy still remains below the oracle upper bound. This shows that label-free selection is improved but remains the central practical limitation.

A second limitation is that the datasets are synthetic and use pre-generated labels. They do not fully represent time-varying wireless channel traces, user mobility, queue backlog, server congestion, multi-cell interference, or multi-server resource competition. Therefore, the trained model may not generalize to realistic MEC environments without further validation.

A third drawback is that the given code does not include full quantitative ablation logs for no clipping, no XLA, fixed heads, loss-term removal, loss-coefficient sweeps, or threshold sensitivity. Instead of generating unsupported ablation results, this thesis sets up an ablation process. Fourth, the repository does not contain Raspberry Pi timing log hence the real edge-device deployment is not verified. The proposed validation process describes the test technique but should not be considered as a conclusive hardware evidence. For some numbers of nodes, the cost scale varies a lot, i.e. raw cost values should not be compared across datasets without further standardization or controlled dataset production.

5.4 Future Work

Future work should improve label-free head selection beyond the basic baselines tested in this revision, including mixture-of-experts routing ideas [19] and edge-inference compression techniques [20]. The most promising direction is to improve the enhanced gating selector with stronger validation design, better calibration, confidence estimate, uncertainty-aware routing, and threshold sensitivity analysis for values like 0.1, 0.3, 0.5, and 0.7. Alternatively, one may utilize mixture-of-experts style routing, uncertainty-aware selection, or reinforcement-learned head selection.

Then the experimental pipeline should extend the controlled ablation framework shown in Tables 4.4 and 4.5 from the example 4-node case to fully recorded runs for all node counts. Each execution should keep the config file, random seed, training log, stored weights, accuracy, mean squared error, cost, latency, and normalized cost improvement. Major improvements include recurring random seeds, larger threshold grids, stronger gating-router validation splits, real device latency measurements.

The model should also be validated using real or semi-real MEC traces and larger multi-server scenarios. To separate synthetic-data performance from deployment performance, the validation approach described in Table 4.12 should be used. Finally, experiments should be run on edge devices such as Raspberry Pi or similar and report the average latency, median latency, 95th percentile latency, memory consumption, model size, and if possible power or thermal performance during sustained inference.

5.5 Final Remarks

The final thesis describes MEMTL as a promising method, although still at the prototype phase. The data, including code, indicates that multi-head specialization can enhance prediction accuracy and MSE with oracle help compared to a single-head baseline. The effort, however, is careful not to overclaim deployment readiness until label free head selection, fully documented ablations, true edge-device measurements, and realistic MEC datasets are incorporated. This prudent interpretation gives credibility to the argument and a more definite direction for future advancement.

References

[1] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A Survey on Mobile Edge Computing: The Communication Perspective," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2322–2358, 2017, doi: 10.1109/COMST.2017.2745201.

[2] P. Mach and Z. Becvar, "Mobile Edge Computing: A Survey on Architecture and Computation Offloading," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 3, pp. 1628–1656, 2017, doi: 10.1109/COMST.2017.2682318.

[3] S. Wang, X. Zhang, Y. Zhang, L. Wang, J. Yang, and W. Wang, "A Survey on Mobile Edge Networks: Convergence of Computing, Caching and Communications," *IEEE Access*, vol. 5, pp. 6757–6779, 2017, doi: 10.1109/ACCESS.2017.2685434.

[4] X. Chen, L. Jiao, W. Li, and X. Fu, "Efficient Multi-User Computation Offloading for Mobile-Edge Cloud Computing," *IEEE/ACM Transactions on Networking*, vol. 24, no. 5, pp. 2795–2808, 2016, doi: 10.1109/TNET.2015.2487344.

[5] S. Bi and Y. J. Zhang, "Computation Rate Maximization for Wireless Powered Mobile-Edge Computing With Binary Computation Offloading," *IEEE Transactions on Wireless Communications*, vol. 17, no. 6, pp. 4177–4190, 2018, doi: 10.1109/TWC.2018.2821664.

[6] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016. [Online]. Available: `https://www.deeplearningbook.org/`

[7] L. Huang, S. Bi, and Y.-J. A. Zhang, "Deep Reinforcement Learning for Online Computation Offloading in Wireless Powered Mobile-Edge Computing Networks," *IEEE Transactions on Mobile Computing*, vol. 19, no. 11, pp. 2581–2593, 2020, doi: 10.1109/TMC.2019.2928811.

[8] M. Chen, Y. Hao, L. Hu, M. S. Hossain, and A. Ghoneim, "Edge-CoCaCo: Toward Joint Optimization of Computation, Caching, and Communication on Edge Cloud," *IEEE Wireless Communications*, vol. 25, no. 3, pp. 21–27, 2018, doi: 10.1109/MWC.2018.1700308.

[9] W. Zhang, Y. Wen, K. Guan, D. Kilper, H. Luo, and D. O. Wu, "Energy-Optimal Mobile Cloud Computing under Stochastic Wireless Channel," *IEEE Transactions on Wireless Communications*, vol. 12, no. 9, pp. 4569–4581, 2013, doi: 10.1109/TWC.2013.072513.121842.

[10] L. Dai, R. Jiao, F. Adachi, H. V. Poor, and L. Hanzo, "Deep Learning for Wireless Communications: An Emerging Interdisciplinary Paradigm," *IEEE Wireless Communications*, vol. 27, no. 4, pp. 133–139, 2020, doi: 10.1109/MWC.001.1900491.

[11] H. Ye, G. Y. Li, and B.-H. F. Juang, "Power of Deep Learning for Channel Estimation and Signal Detection in OFDM Systems," *IEEE Wireless Communications Letters*, vol. 7, no. 1, pp. 114–117, 2018, doi: 10.1109/LWC.2017.2757490.

[12] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738–1762, 2019, doi: 10.1109/JPROC.2019.2918951.

[13] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018. [Online]. Available: http://incompleteideas.net/book/the-book-2nd.html

[14] R. Caruana, "Multitask Learning," *Machine Learning*, vol. 28, no. 1, pp. 41–75, 1997, doi: 10.1023/A:1007379606734.

[15] S. Ruder, "An Overview of Multi-Task Learning in Deep Neural Networks," arXiv preprint arXiv:1706.05098, 2017. [Online]. Available: https://arxiv.org/abs/1706.05098

[16] Y. Zhang and Q. Yang, "A Survey on Multi-Task Learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 12, pp. 5586–5609, 2022, doi: 10.1109/TKDE.2021.3070203.

[17] L. Rokach, "Ensemble-Based Classifiers," *Artificial Intelligence Review*, vol. 33, no. 1–2, pp. 1–39, 2010, doi: 10.1007/s10462-009-9124-7.

[18] T. G. Dietterich, "Ensemble Methods in Machine Learning," in *Multiple Classifier Systems*, Berlin, Germany: Springer, 2000, pp. 1–15, doi: 10.1007/3-540-45014-9_1.

[19] N. Shazeer et al., "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer," in *Proc. International Conference on Learning Representations*, 2017. [Online]. Available: https://arxiv.org/abs/1701.06538

[20] S. Han, H. Mao, and W. J. Dally, "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding," in *Proc. International Conference on Learning Representations*, 2016. [Online]. Available: https://arxiv.org/abs/1510.00149

[21] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems," arXiv preprint arXiv:1603.04467, 2016. [Online]. Available: https://arxiv.org/abs/1603.04467

[22] OpenXLA Authors, "XLA: Optimizing Compiler for Machine Learning," OpenXLA Documentation. [Online]. Available: https://openxla.org/xla/tf2xla. Accessed: May 24, 2026.

[23] Keras Developers, "Keras 3 API Documentation," Keras Documentation. [Online]. Available: https://keras.io/api/. Accessed: May 24, 2026.

[24] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proc. International Conference on Learning Representations*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980

[25] F. Chollet, *Deep Learning with Python*, 2nd ed. Shelter Island, NY, USA: Manning, 2021. [Online]. Available: https://www.manning.com/books/deep-learning-with-python-second-edition

Chapter A Core Implementation and Reproducibility Notes

A.1 Main Code Modules

Implementation is organized into modules for model, utility, example and configuration. MEMTL model file describes the shared backbone, numerous prediction heads, bespoke training step, dynamic head expansion methods and ensemble inference logic. The loss-function module implements the three-part training objective . The dataset module loads the CSV datasets and splits them in training, testing and head-specific subspaces. The experiment scripts replicate baseline MTL-FNN training, MEMTL multi-stage training, oracle-assisted evaluation, and TensorFlow/XLA latency measurement.

A.2 Reproducibility Checklist

To reproduce the experiments, the random seed, dataset size, node number, learning rate, batch size, gradient clipping norm, decision threshold, number of prediction heads, and XLA setting should be fixed in the configuration files. For latency evaluation, one or more warm-up iterations should be excluded from the timing result, and the reported latency should be averaged over repeated measurements on the same hardware platform.

The head-pick-rate diagnostic in Table 4.9 should be reproduced by loading the saved 5-head H5 weights, applying the same MinMax scaling, using the sorted 10,000-sample test split, running all five heads, and applying the same oracle selection rule used in the experiment script.